

Programming Languages & Compilers

Lecture Notes Summary

AIT Masters in Computer Science — Year 1, Semester 2

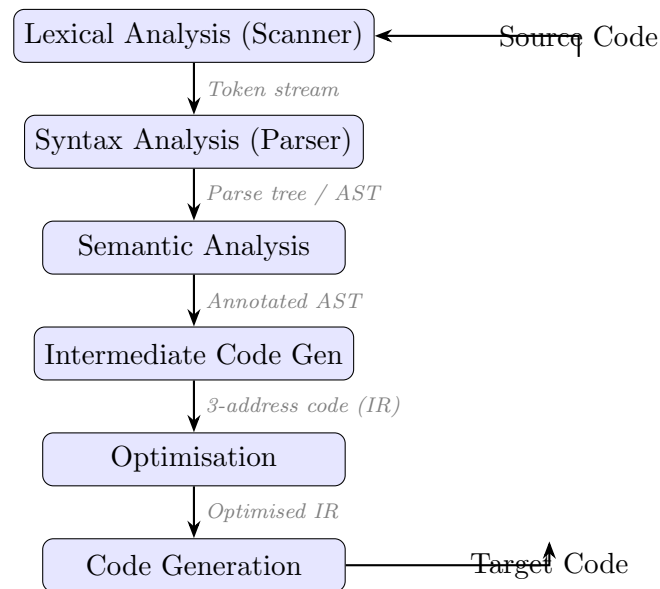
Study Notes

2026

Contents

1 Compiler Overview

A compiler translates source code through a sequence of well-defined phases. Each phase transforms one representation into a lower-level, more machine-friendly one.



2 Lexical Analysis

The **scanner** reads the character stream and groups characters into *tokens* (terminal symbols of the grammar).

Token A pair $\langle \text{class}, \text{value} \rangle$, e.g. $\langle \text{NUM}, 30 \rangle$.

Lexeme The actual character sequence matched, e.g. "30".

Pattern A regular expression describing valid lexemes for a token class.

2.1 Tokens and Patterns

Token	Pattern (regex)	Example lexeme
NUM	$[0-9]^+$	30
ID	$[a-zA-Z_][w]^*$	count
PLUS	$\backslash +$	+
TIMES	$\backslash *$	*

2.2 DFA for NUM and *

The scanner is implemented as a DFA. Below is the combined DFA recognising **NUM** (one or more digits) and **TIMES** (a single *).

2.3 Lexical Analysis Trace: "30 * 4 \$"

Input consumed	Token emitted	Class	Value
30	$\langle \text{NUM}, 30 \rangle$	NUM	30
*	$\langle \text{TIMES}, * \rangle$	TIMES	*
4	$\langle \text{NUM}, 4 \rangle$	NUM	4
\$	end-of-input	—	—

2.4 Pipeline: RegEx $\rightarrow$ NFA $\rightarrow$ DFA $\rightarrow$ Minimised DFA

1. Write a regular expression for each token class.
2. Build an NFA via *Thompson's construction*.
3. Convert NFA $\rightarrow$ DFA via *subset (powerset) construction*.
4. Minimise the DFA via *Hopcroft's algorithm*.
5. Run the DFA on input; *longest match* wins.

3 Syntax Analysis (Parsing)

The **parser** checks that the token stream conforms to the grammar and builds a parse tree or AST.

3.1 Context-Free Grammars

A CFG is a 4-tuple $G = (V, T, P, S)$ where V is the set of non-terminals, T is the set of terminals, P is the set of productions, and S is the start symbol.

Example grammar for arithmetic expressions:

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid \mathbf{num}$$

3.2 Grammar Classification

Class	Strategy	Lookahead	Tool
LL(1)	Top-down, predictive	1 token	Recursive descent
LR(0)	Bottom-up, shift-reduce	0	Simplest SLR
SLR(1)	Bottom-up, FOLLOW sets	1	SLR
LALR(1)	Bottom-up, lookahead	1	yacc/bison (default)
Canonical LR(1)	Full LR, largest tables	1	Most powerful

3.3 LL(1) Parsing

FIRST(α) Set of terminals that can begin strings derived from α .

FOLLOW(A) Set of terminals that can appear immediately after A .

Parse table entry:

$$M[A, a] = A \rightarrow \alpha \quad \text{if } a \in \text{FIRST}(\alpha), \quad M[A, a] = A \rightarrow \alpha \quad \text{if } \varepsilon \in \text{FIRST}(\alpha) \text{ and } a \in \text{FOLLOW}(A).$$

3.4 Shift-Reduce (LR) Parsing

The parser maintains a *stack* and inspects the remaining *input*:

Shift Push the next input token onto the stack.

Reduce Pop the *handle* β from the stack; push LHS non-terminal A (for production $A \rightarrow \beta$).

Accept Input fully consumed and stack contains only the start symbol.

Error No valid action.

Conflicts:

- *Shift-Reduce conflict*: ambiguous whether to shift the next token or reduce.
- *Reduce-Reduce conflict*: two productions can reduce the same handle.

3.4.1 LR(0) Items and Closure

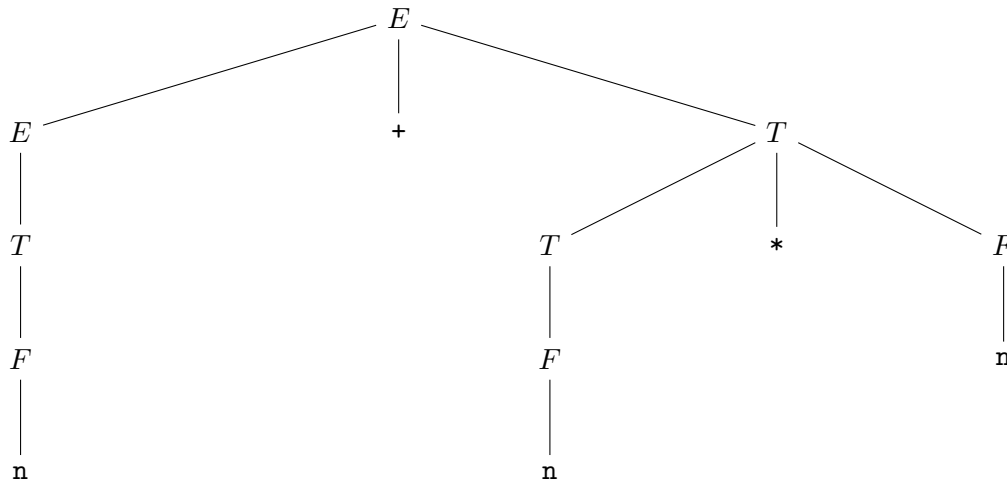
An **LR(0) item** is a production with a dot marking parse progress:

$$E \rightarrow E + \bullet T \quad (\text{expecting } T \text{ next}) \quad E \rightarrow E + T \bullet \quad (\text{ready to reduce})$$

Closure(I): for every item $[A \rightarrow \alpha \bullet B\beta]$ in I , add all items $[B \rightarrow \bullet \gamma]$.

Goto(I, X): move the dot past X in every applicable item of I .

3.5 Parse Tree Example: $n + n * n$



4 The AIT Acceptor (PDA-based LR Parser)

For languages that require $LR(n > 1)$ lookahead, the course introduces the **AIT acceptor**, a pushdown automaton tailored to shift-reduce parsing.

4.1 Definition

An AIT acceptor is a **sextuple**

$$M = (K, \Sigma, \Gamma, \Delta, s, F)$$

where

K Finite set of states.

Σ Input alphabet.

Γ Stack alphabet.

Δ Set of transitions $(q, a, \beta) \rightarrow (q', \gamma, i)$ where $i \in \{0, 1\}$:
 $i = 1$ consume input symbol; $i = 0$ do not consume input.

s Start state.

F Set of accepting states.

4.2 Configurations

A **configuration** is a triple (q, w, v) :

- $q \in K$ — current state.
- $w \in \Sigma^*$ — remaining input.
- $v \in \Gamma^*$ — current stack contents (top at left).

Initial configuration: (s, w_0, ε) .

Acceptance: $(q_f, \varepsilon, \varepsilon)$ for some $q_f \in F$.

4.3 Example: Balanced Parentheses

Grammar: $S \rightarrow (S) \mid SS \mid \varepsilon$. Step-by-step trace on input $(())$:

State	Input remaining	Stack	Action
q_0	$(())$	ε	Shift (
q_0	$()$	(	Shift (
q_0	$)$	((	Shift)
q_0	$)$	(	Reduce $S \rightarrow (S)$
q_0	$)$	S	Shift)
q_0	ε	S	Reduce $S \rightarrow (S)$
q_f	ε	ε	Accept

5 Values and Types

A **value** is anything that can be evaluated, stored, incorporated in a data structure, passed as an argument, or returned as a function result.

Values are grouped into **types** — sets of values sharing characteristic operations. Types may be *primitive*, *composite*, or *recursive*.

5.1 Primitive Types of Pascal

Integer Range $-32768 \dots 32767$. Operations: $+$, $-$, $\times$, $\div$, **mod**; conversion to real. Relations: $<$, $\leq$, $>$, $\geq$, $=$.

Real Single and double precision.

Char ASCII characters. Operations: *Ord*, *Chr*, *Pred*, *Succ*.

Boolean Values: *True*, *False*. Logical: *not*, *and*, *or*. Ordinal: $Ord(\text{False}) = 0$, $Ord(\text{True}) = 1$.

Enumerated User-defined ordinal type. `type Day = (Sunday, Monday, ..., Saturday);`

Subrange Subset of an ordinal host type. `type Lowercase = 'a'..'z';`

5.2 Composite Types of Pascal

Composite types are built via set-theoretic operations: Cartesian products, disjoint unions, powersets, and function (mapping) types.

5.2.1 Record (Cartesian Product)

```
type Employee = record
    Age      : Integer;
    Gender   : Char;
end;
```

Values are n -tuples: $\text{Integer} \times \text{Char}$.

5.2.2 Variant Record (Disjoint Union)

```
type Accuracy = (exact, appro);
Number       = record
    case acc : Accuracy of
        exact : (ival: Integer);
        appro : (rval: Real);
    end;
```

The tag field *acc* determines which variant is active.

5.2.3 Set (Powerset)

```
type settype = set of basetype;    { basetype must be ordinal }
```

Operations: $\in$ (**in**), union (+), intersection (*), difference (−).

5.2.4 Array (Function Type)

```
type arraytype = array[subscripttype] of elementtype;
```

Values are functions $\text{subscripttype} \rightarrow \text{elementtype}$. The subscript type must be a finite ordinal type.

5.3 Types in Camllight

Integer Range $-2^{30} \dots 2^{30} - 1$, arithmetic wraps modulo 2^{31} . No operator overloading; integer and float operators are distinct.

Float Separate operators: $+. , -. , *. , /. , \text{sqrt}$, trig functions, `int_of_float/float_of_int`.

Product types (tuples) $t * t'$ defined automatically. Constructor `(,)`; projection `fst/snd`.

Generic abstraction Parameterised over types: `#type ('a,'b) pair = {Fst: 'a; Snd: 'b};;`

6 Type Checking

A **type system** assigns types to expressions and enforces type safety.

6.1 Static vs. Dynamic Typing

Static typing Types checked at compile time. Languages: Pascal, Java, ML, Camllight.
Catches type errors before execution.

Dynamic typing Types checked at runtime. Languages: Lisp, Smalltalk. More flexible but errors only discovered at execution.

Rule: Languages with static typing adopt *static binding*; languages with dynamic typing adopt *dynamic binding*. Mixing static typing with dynamic binding breaks the static type checker.

6.2 Type Expressions (Simple Language)

Type expressions for Simple Language 1:

$$\tau ::= \text{int} \mid \text{real} \mid \text{bool}$$

6.3 Type Checking Rules

The function `addtype(id, type)` inserts a new type binding into the symbol table. The function `lookup(id)` retrieves the type.

Inference rule for addition:

$$\frac{\Gamma \vdash e_1 : \text{int} \quad \Gamma \vdash e_2 : \text{int}}{\Gamma \vdash e_1 + e_2 : \text{int}} \quad (\text{T-Plus})$$

6.4 Type Equivalence

Name equivalence Two types are equal iff they have the *same name*. Used in Pascal for record types.

Structural equivalence Two types are equal iff they have the *same structure*. Used for arrays and pointers in Pascal.

7 Syntax-Directed Translation (SDT)

In SDT each grammar symbol carries one or more **attributes**. Semantic rules attached to productions define how attribute values are computed.

7.1 Attribute Types

Synthesised Computed *bottom-up* from children.

Inherited Passed *top-down* from parent or siblings.

7.2 Ambiguity and Associativity

The grammar $E \rightarrow E - E \mid \mathbf{num}$ is ambiguous. Ambiguity affects evaluation order:

- Left-associative: $10 - 5 - 2 = (10 - 5) - 2 = 3$.
- Right-associative: $10 - 5 - 2 = 10 - (5 - 2) = 7$.

Unambiguous left-recursive grammar: $E \rightarrow E - T \mid T, T \rightarrow \mathbf{num}$.

7.3 Prefix and Postfix Notation

For the expression $10 - 5 - 2$ (left-associative):

- **Prefix:** $- - 10 5 2$
- **Infix:** $10 - 5 - 2$
- **Postfix:** $10 5 - 2 -$

The **prefix attribute** $E.prf$ is a synthesised attribute built bottom-up:

$$E \rightarrow E_1 - T \quad E.prf := "-" \parallel E_1.prf \parallel T.prf$$

7.4 Appendix: Recursive Top-Down Parser-Translator

A translation scheme embeds semantic actions within grammar productions. The scheme can be executed directly as a recursive descent parser-translator.

Example grammar with embedded actions:

$$\begin{aligned} E &\rightarrow T R \\ R &\rightarrow \mathbf{addop} T \{ \text{print}(\mathbf{addop.lexeme}) \} R_1 \mid \varepsilon \\ T &\rightarrow \mathbf{num} \{ \text{print}(\mathbf{num.lexeme}) \} \end{aligned}$$

A left-recursive SDT grammar with synthesised attribute $E.val$:

$$E \rightarrow E_1 + \mathbf{num} \{ E.val := E_1.val + \mathbf{num.lex} \}$$

is transformed by eliminating left recursion, introducing inherited attribute $E'.i$ and output attribute $E'.o$, then converted to a recursive top-down translator.

8 Intermediate Code Generation (Three-Address Code)

Three-address code (3AC) has at most one operator per statement.

8.1 Five Forms of 3AC Statements

1. $x := y \mathbf{op} z$ (binary operation)
2. $x := \mathbf{op} y$ (unary operation)
3. $x := y$ (copy)
4. $\mathbf{goto} L$ (unconditional jump)
5. $\mathbf{if} x \mathbf{relop} y \mathbf{goto} L$ (conditional jump)

8.2 Syntax-Directed Code Generation

Each non-terminal E carries two attributes:

$E.place$ The name of the variable or temporary holding the value.

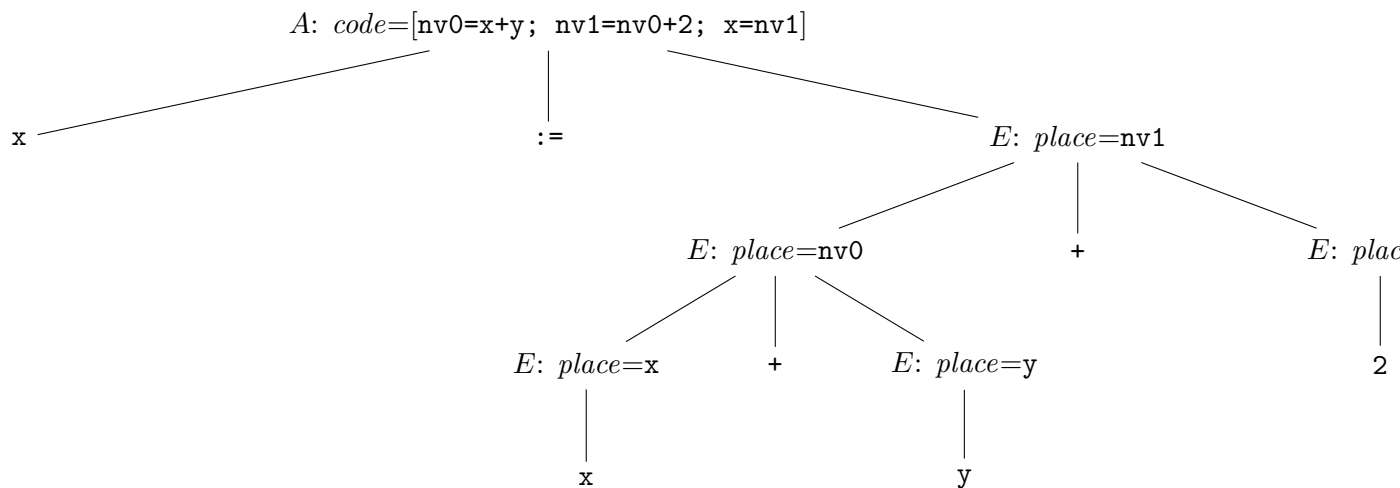
$E.code$ The sequence of 3AC statements to compute the value.

The function `new_var()` generates a fresh temporary variable name.

Semantic rules for Simple Language (key rules):

Production	Semantic rule
$A \rightarrow \text{id} := E$	$A.code = E.code \parallel \text{id}.place := E.place$
$E \rightarrow E_1 + E_2$	$E.place = \text{new_var}()$ $E.code = E_1.code \parallel E_2.code \parallel E.place := E_1.place + E_2.place$
$E \rightarrow \text{num}$	$E.place = \text{num.val}; \quad E.code = \varepsilon$
$E \rightarrow \text{id}$	$E.place = \text{id.name}; \quad E.code = \varepsilon$

8.3 Annotated Parse Tree Example: $x = x + y + 2$



Generated 3AC:

```

nv0 := x + y
nv1 := nv0 + 2
x   := nv1

```

9 Storage and Variables

9.1 The Store Model

A **store** is a collection of *cells* (locations). Each cell is either *allocated* or *unallocated*. An allocated cell contains either a *storable value* or is *undefined*.

Variable declaration in Pascal: `var n: Integer` has two effects: (1) an unallocated cell is found and *bound* to n ; (2) its status becomes allocated with content *undefined*.

At the end of the block containing the declaration, the cell reverts to unallocated.

9.2 Storable Values

Storable values are values that can be stored in a single cell.

Language	Storable values
Pascal	Primitive values, sets, pointers
Java	All primitive values and object references (arrays are objects)

The design of what is storable profoundly affects language compactness.

9.3 Composite Variables

A composite variable is a collection of variables whose contents form the components of a composite value.

```
type Employee = record Age: Integer; Gender: Char; end;
var Peter, Copy: Employee;
Peter.Age := 21; Peter.Gender := 'M';
Copy := Peter;    { shorthand for: Copy.Age:=Peter.Age; Copy.
                  Gender:=Peter.Gender }
```

Pascal vs. Java arrays:

- **Pascal:** `Y := X` copies the array value (value semantics). After `Y[0] := 5`, `X[0]` remains 1.
- **Java:** `int[] Y = X` copies the *reference*. After `Y[0] = 5`, `X[0]` is also 5 (alias).

9.4 Lifetime of Variables

The **lifetime** of a variable is the interval between its creation and deletion.

Local variable Created for use only within a block; destroyed at block exit.

Global variable Created in the outermost block; lives for the entire program.

9.5 Dangling References

A **dangling reference** occurs when a pointer refers to a cell that has been deallocated (via **dispose** in Pascal).

```
new(X); new(Y); X^ := 2; Y^ := X; Writeln(Y^^);
dispose(X);    { cell X pointed to is freed }
new(Z); Z := 4;
Writeln(Y^^); { DANGER: Y still points to the freed cell, now
                reused by Z }
```

Java avoids dangling references by preventing explicit deallocation; a *Garbage Collector* automatically disposes unreachable objects.

9.6 Commands

Commands update the store. Basic kinds: *skip*, *assignment*, *procedure calls*, *sequential*, *iterative*, *expressions with side-effects*.

10 Bindings and Environments

10.1 Bindings and Environments

A **binding** associates an identifier with an entity; it is produced by a *declaration*.

An **environment** is a set of bindings where each identifier is bound to at most one entity.

Example — Pascal program environment at two points:

At point (1) (main body): $\{c \mapsto \text{cell}_{\text{char}}, q \mapsto \text{proc-abstraction}, z \mapsto 0\}$.

At point (2) (inside procedure Q): $\{b \mapsto \text{cell}_{\text{bool}}, c \mapsto 2.5, q \mapsto \text{proc-abstraction}, z \mapsto 0\}$.

10.2 Bindable Entities

Language	Bindable entities
Pascal	Primitive values, strings, locations, procedure/function abstractions, types. <i>Note:</i> Composite values (records, sets, arrays) are <i>not</i> bindable.
Camllight	Primitive values, composite values, procedure/function abstractions, types, exceptions.
Java	Primitive values, objects, locations, object methods, classes, interfaces, exceptions.

10.3 Scope

Each declaration has a **scope** — the block in which it is visible.

Binding occurrence Where the identifier is declared (`var x: Integer`).

Applied occurrence Where the identifier is used (`y := x + 1`).

If the same identifier is declared in two nested blocks, the inner declaration *shadows* (hides) the outer one.

10.4 Static vs. Dynamic Binding

Static binding The body of a function/procedure abstraction is evaluated in the *environment of its definition* (lexical scope). Used by Pascal, ML, Java.

Dynamic binding The body is evaluated in the *environment of the call site*. Used by Lisp, Smalltalk.

Key interaction: Static typing + dynamic binding is problematic. Under dynamic binding, type correctness of a function call depends on which binding of free variables is

in scope at call time — this can only be checked at runtime, defeating the purpose of a static type checker.

Dynamic binding example in Clisp:

```
> (defvar s 2)
> (defun f(d) (* d s))
> (defun g() (let ((s 3)) (f 3))) ; rebinds s to 3 locally
> (f 3) ; => 6 (uses s=2)
> (g) ; => 9 (dynamic: f sees s=3 from g's environment)
```

10.5 Declarations and Blocks

Block commands A command containing a declaration whose bindings are used only for that command.

Block expressions An expression containing a declaration whose bindings are used only for that expression.

Block declarations A declaration containing local declarations whose bindings are used only for elaborating that declaration.

The Qualification Principle: Blocks are allowed for expressions, commands, and declarations. This principle is the cornerstone of information hiding (encapsulation). Java realises it through `class` with `public/private` modifiers.

11 Abstraction

Abstraction distinguishes *what* a program does from *how* it is implemented.

11.1 Function Abstraction

A **function abstraction** abstracts over an expression; when called, it returns a value.

Camllight:

```
# let rec power = function x, n ->
    if n = 1 then x else x * power(x, n-1);;
```

Pascal:

```
function power(x: Integer; n: Integer): Integer;
begin
    if n = 1 then power := x
    else power := x * power(x, n-1)
end;
```

Note: In Pascal, the identifier `power` denotes both a variable (for returning the result) and the function abstraction in the same scope, which can cause confusion.

11.2 Procedure Abstraction

A **procedure abstraction** abstracts over a *command* for updating variables. The caller observes only the effects of updating variables, not the execution details.

11.3 Generic Abstraction

Generic abstraction parameterises over types. Pascal requires fixed types for parameters; Camllight supports polymorphism:

```
# type ('a, 'b) pair = {Fst: 'a; Snd: 'b};;
```

11.4 Parameter Mechanisms

When an abstraction is called, actual parameters are passed to formal parameters.

11.4.1 Copying Mechanism

Value parameter A local variable is created and assigned the argument value on entry.

Result parameter A local variable is created; on exit its value is assigned back to the argument variable.

Value-Result parameter Combination: copy in on entry, copy out on exit.

11.4.2 Definitional Mechanism

Constant parameter Formal parameter is bound directly to the argument value.

Variable parameter (var) Formal parameter is bound to the argument variable (i.e. the *location*, not the value). **Aliasing risk:** `confuse(i, i)` with `var m, n` makes *m* and *n* aliases for the same cell.

Function/Procedure parameter Formal parameter is bound to a function/procedure abstraction.

Camllight uses definitional mechanism only. Pascal uses both value parameters and variable (`var`) parameters.

11.5 The Correspondence Principle

For each form of declaration there is a corresponding form of parameter mechanism, and vice versa.

Camllight satisfies this principle (variable parameter $\leftrightarrow$ `let s = r;;`). Pascal *does not*: there is no declaration with the same effect as Pascal's `var` parameter.

11.6 Evaluation Order

Eager evaluation Actual parameters are evaluated *once* before the abstraction body executes. `sqr(p + q)` with $p = 2, q = 5$: evaluate $p + q = 7$ first, then $7 * 7 = 49$.

Normal evaluation Formal parameter is bound to the *unevaluated expression*; evaluated each time it is needed. `sqr(p + q)`: compute $(2 + 5) * (2 + 5) = 49$.

For pure functional programs (no side effects), both strategies yield the same result (**Church-Rosser Property**): *If an expression can be evaluated at all, normal-order evaluation will find the result; all terminating strategies yield the same value.*

For expressions with side effects (Pascal, Camllight), evaluation order matters:

```
# let x = ref 1;;
# let f = function () -> x := !x + 1; !x;;
# f() + !x;;      (* evaluates right-to-left in Camllight: 3 *)
# !x + f();;      (* evaluates right-to-left: 4 *)
```

Camllight uses **eager (right-to-left) evaluation**.

12 Summary Table

Feature	Pascal	Camllight	Java
Typing	Static	Static	Static
Binding	Static	Static	Static
Storable composites	No	Yes (tuples)	Yes (objects)
Generic abstraction	No	Yes	Yes (generics)
Parameter mechanism	Value + Var	Definitional	Value (refs for objects)
Garbage collection	No	Yes	Yes
Correspondence principle	No	Yes	Partial
Evaluation order	Left-to-right	Right-to-left	Left-to-right